

AI-Assisted Development Prompt Sequence

Playpower Labs — Airbnb Clone Assessment

Section 1 — Assignment Analysis and Clean Workspace

- Read the assignment PDF completely and carefully before starting development. For this first step, do not create frontend or backend code. Create only `ASSIGNMENT_ANALYSIS.md` and record requirements that are explicitly supported by the assignment. Label each item as `REQUIRED`, `OPTIONAL`, `NOT REQUIRED`, or `RESTRICTION`. Do not add assumptions or generic Airbnb requirements. If something is not specified, explicitly write: "Not specified in the assignment PDF." Do not use the reference website or search the internet during this step. After the analysis is created, show the final folder structure and the complete file contents, then stop.
- Keep the workspace clean after the analysis step. Only keep the original assignment PDF and `ASSIGNMENT_ANALYSIS.md` for now. Do not begin implementation until the project requirements are clear.

Section 2 — Initial Implementation from Visual References

- I am starting the Airbnb-style clone now. Use the supplied reference screenshots as the visual direction and build the page structure around the assignment requirements. Keep the implementation practical for an assessment project: clear components, readable styling, and no unnecessary architecture. Match the spacing, layout hierarchy, navigation, property information, gallery, and booking sections shown in the references as closely as practical.
- Work section by section rather than changing the entire application at once. After each major change, keep the existing layout working and avoid unrelated refactoring. The goal is to get a close visual match first, then refine individual interactions and edge cases.
- I will provide screenshots of specific sections when something is not matching the reference. Update only the relevant section, preserve the rest of the page, and make the smallest change needed to keep the behaviour consistent.

Section 3 — Main Header and Navigation Refinement

- Update the right side of the main header navigation. The language selector and profile/menu control should look like clean circular or pill-shaped Airbnb-style actions. Use subtle hover states, proper spacing, and cursor feedback. The profile control should contain the hamburger icon and a user/avatar icon.
- The "Become a host" navigation item is already present. Keep it functional and route it to a simple placeholder route such as `/host` using the existing React Router setup. For the language and profile buttons, add lightweight state so a simple dropdown or menu can open on click. Do not build a large menu system yet.
- Check the header against the supplied screenshot. Fix icon size, spacing, borders, hover behaviour, and alignment without changing unrelated sections of the page.

Section 4 — Sticky Property Navigation and Smooth Scrolling

- The main header should scroll away normally. Remove sticky/fixed behaviour from the top navbar so it remains part of the normal page flow.
- Add a secondary navigation bar directly below the hero photo gallery. Once the user scrolls past the photo section, this secondary bar should stick to the top of the viewport. Use a white background, subtle bottom border, shadow, and a high enough z-index so the navigation remains visible while scrolling.
- The secondary navigation should contain Photos, Amenities, Reviews, Location. Clicking a section should smoothly scroll to the relevant DOM element using `scrollIntoView({ behavior: 'smooth' })`. Add a simple active state using a darker bottom border or stronger font weight.

- On the right side of the sticky navigation, show a compact pricing summary such as “■28,499 for 5 nights” with the rating “★ 4.95 · 19 reviews” and a primary pink/red Reserve button. Keep the bar responsive so it does not break on smaller screens.

Section 5 — Sticky Reservation Box and Split-Scroll Layout

- The reservation box should remain visible while the user scrolls through the content on the left. Use a sticky positioning approach so the right-side reservation card stays fixed within the viewport as the left-side content continues to scroll naturally.
- The sticky behaviour should begin after the main hero section and should remain active only within the property details section. Once the user reaches the end of that section, the reservation box should stop rather than overlapping or moving into the sections below.
- Keep the reservation box visually consistent with the supplied Airbnb reference, including its spacing, border, shadow, pricing information, guest/date controls, and Reserve button. Make sure the two-column layout remains responsive and that the sticky behaviour does not break on smaller screens.

Section 6 — Photo Tour and Category-Based Navigation

- Add a “Show more photos” / photo tour section similar to the reference screenshots. At the top, show the main photo categories or section headings. The category thumbnails should be clickable and should grow slightly on hover so the user can tell they are interactive.
- Within the photo tour, group images into categories such as Living room, Bedroom, Kitchen, Exterior, View, and other relevant property areas. Under each category, show a main/featured image followed by additional angles. Keep the category label visible while its images are being viewed, and make the section navigation clear when scrolling between categories.
- Use the supplied visual references for the overall composition, spacing, image proportions, and typography. Extra images can be added where the reference shows a fuller gallery, but do not redesign the page into something unrelated to the given layout.
- When a user clicks a photo from the main property page, take them to the corresponding location in the photo tour instead of opening a different unrelated gallery. Preserve which category and subsection the selected image belongs to.

Section 7 — Full-Screen Photo Viewer and Keyboard Interaction

- When a photo is clicked inside the photo tour, open a full-screen image viewer showing the photos one by one. The viewer should use the same set of images that belong to the photo tour and should work regardless of which image was clicked first.
- The viewer should support moving to the previous and next image and should also work with the keyboard. Keep the interaction lightweight and predictable: users should be able to navigate without having to close and reopen the viewer.
- In the single-photo view, place the current photo category name at the top centre and show the current image number / total count at the top right. The category label should update as the user moves between categories.
- If a photo is opened from the main property gallery, start the viewer at that exact photo rather than always starting from the first image. Do not change other property-page behaviour.

Section 8 — Backend API, Fallback Data, and Environment Variables

- For the backend, keep the solution intentionally small because this is an assessment project. Use a basic Express server with a listing endpoint and a simple JSON data source. The frontend should fetch the listing data through the API rather than hard-coding every value directly in the component.

- Add sensible error handling around the API request. If the API is temporarily unavailable during local development, allow the frontend to fall back to the existing local/mock listing data so the UI can still render. Do not let this fallback replace the normal API path.
- Check the project for any environment variables or API URLs that may have been written directly into the code. Move values such as the backend base URL into the appropriate environment configuration and read them through the Vite environment mechanism instead of hard-coding localhost URLs in components.

Final Section — Responsive Fixes and Final Polish

- Check the current implementation after the latest change and fix any visual or functional issues that appeared. Do not rewrite unrelated components. Preserve the parts of the page that already match the reference.
- Before finalizing, check for unnecessary hard-coded values, broken links, console errors, missing assets, unused imports, incorrect environment-variable references, and UI states that behave differently from the supplied references.

Note — Why the Prompts Are Consolidated

The actual development process included use of more than one AI tool, many short follow-up messages, and small corrections. For submission, those messages are grouped by feature so the document remains readable and still represents the real progression of the work. The wording has been cleaned up for readability, but the technical intent and order are preserved.